dnsflowd

Live DNS Telemetry & Lightweight Blocking

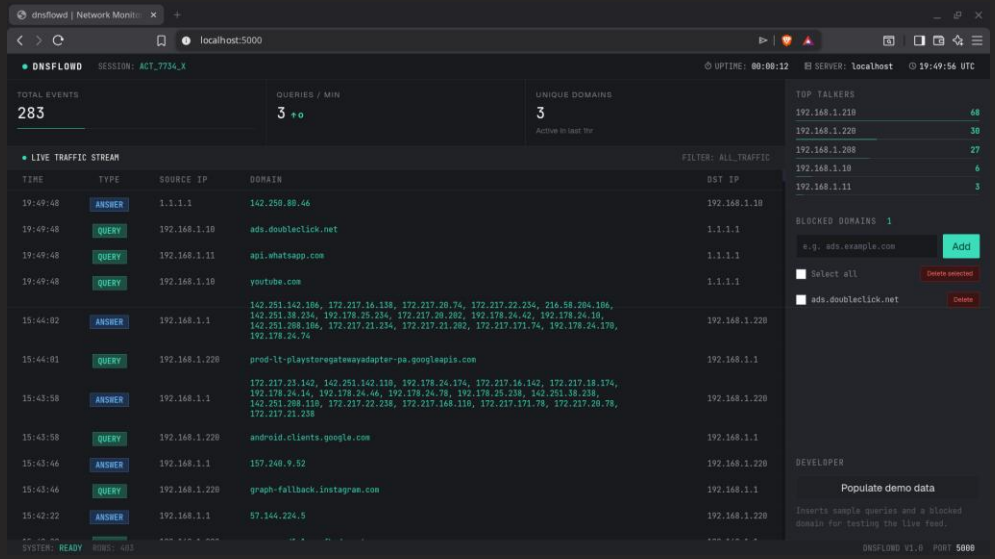
A minimal, end-to-end pipeline for home and edge networks: **capture → transport → store → display → act**. Built on OpenWrt, Python, SQLite, and Flask — designed to be simple, teachable, and immediately useful.

OPENWRT | PYTHON | SQLITE | FLASK + SSE

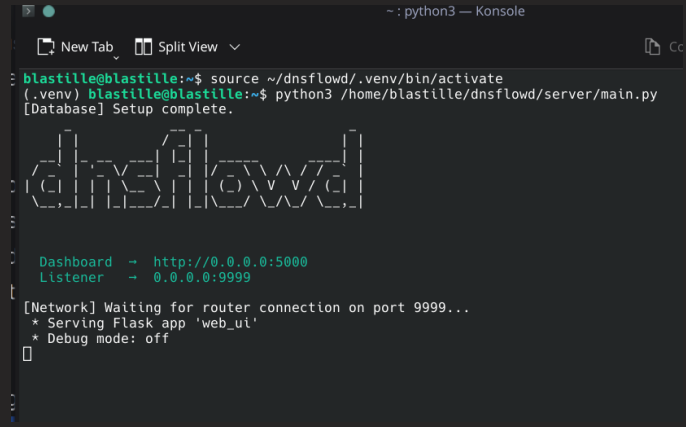
- Abdulrahman Abdulkader
- Abdulrahman Al-Muhammad
- Abdulaziz Haji-Bakkour
- Issam Al-Maarouf
- Muhammad Sheikho

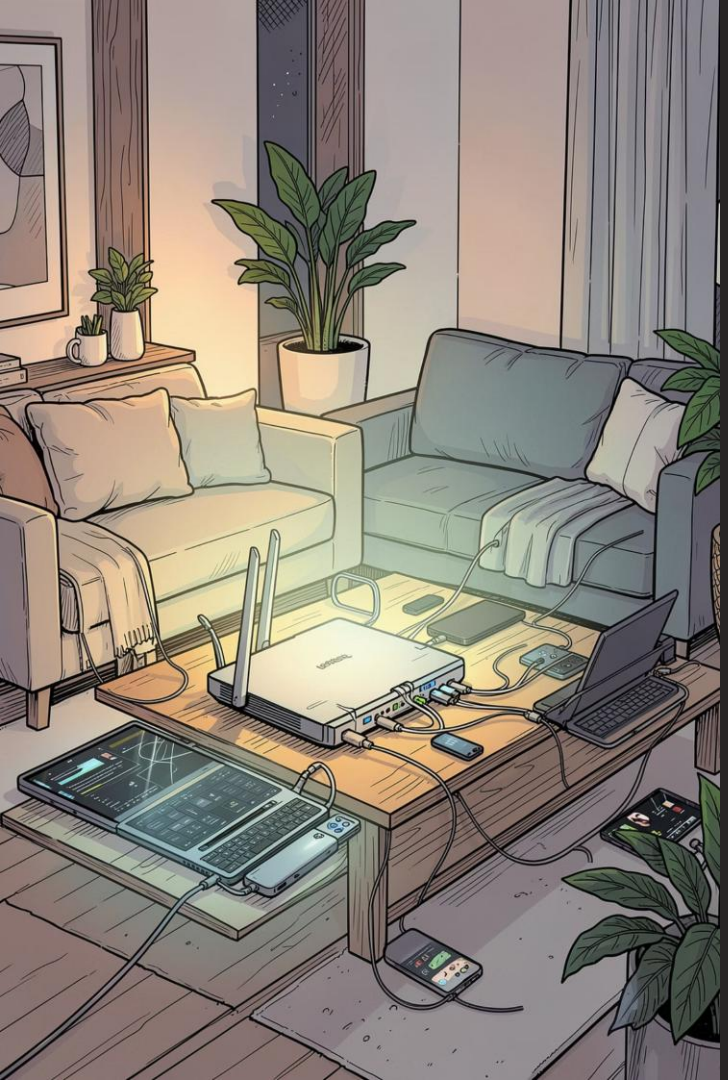
User Interfaces

Web UI



CLI





Why DNS Telemetry?

DNS is the phone book of the internet. Every device consults it before connecting anywhere. Watching that traffic in real time gives you visibility into what your devices are doing, without deep packet inspection or heavyweight tooling.

🔍 Visibility

Monitor DNS queries per device in real time — see which hosts are talking to which domains.

🛡️ Security

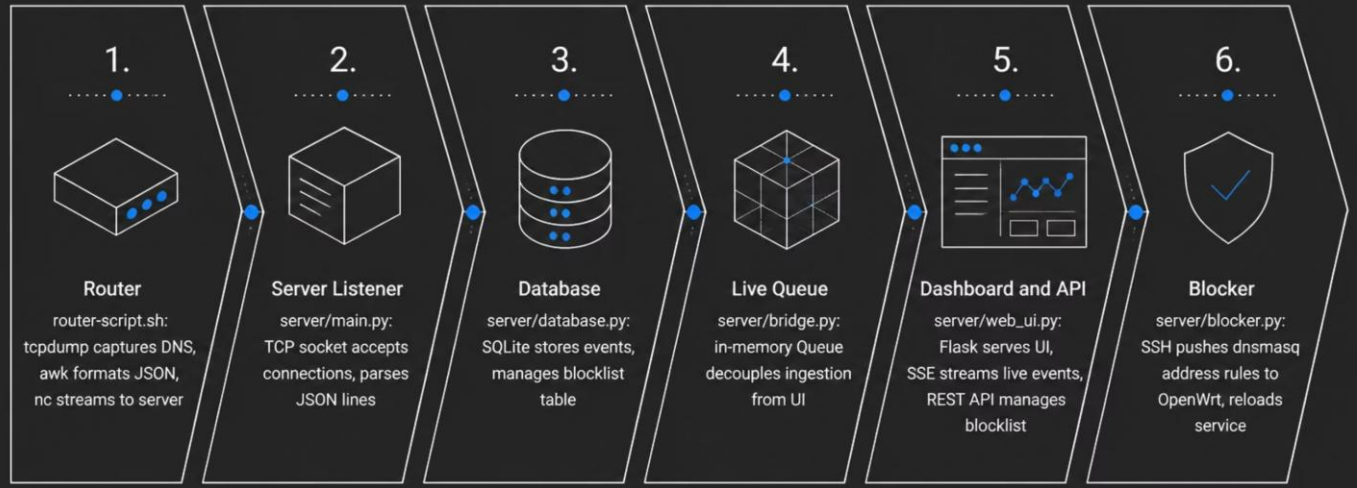
Centralize blocking for ads, trackers, and malware at the network edge — no per-device config needed.

🧩 Teachable

Demonstrate streaming ingestion, queuing, persistence, and a live web UI with minimal, readable code.

High-Level Architecture

Six small, decoupled components form the full pipeline. The router produces newline-delimited JSON; the server ingests, persists, and pushes live events to the dashboard. The blocker closes the loop by writing dnsmasq rules back to the router.



Algorithms Used in dnsflowd

Three core algorithms power the system:

Hashmaps (Dictionaries) — O(1) average-time counting

Used in `server/analytics.py` to count how often each source IP appears in recent queries. Fast lookups and updates enable real-time frequency analysis.

```
# server/analytics.py
frequency_map = {}
for query in data:
    src = query.get("src_ip")
    if src in frequency_map:
        frequency_map[src] += 1
    else:
        frequency_map[src] = 1
```

Trees & Indexing — O(log n) search

SQLite indexes in `server/database.py` speed up searches on `detail`, `src_ip`, and blocked domains. Multi-column indexes enable efficient range queries and joins.

```
# server/database.py
cur.execute("CREATE INDEX IF NOT
EXISTS idx_detail_src ON dns_traffic
(detail, src_ip)")
cur.execute("CREATE INDEX IF NOT
EXISTS idx_blocked_domain ON
blocked_domains (domain)")
```

Sorting — O(n log n) ranking

Sorts the frequency map to rank the most active clients by request count and order recent records for dashboard display.

```
frequency_list =
list(frequency_map.items())
frequency_list.sort(key=lambda item:
item[1], reverse=True)
top_talkers = frequency_list[:10]
```

Why it matters: Hashmaps give us instant counting, indexes let us query millions of DNS events without full table scans, and sorting surfaces the most interesting data for the UI.

Router Capture Script

ROUTER-SCRIPT.SH

A single shell script on OpenWrt captures DNS packets with `tcpdump`, parses them with `awk` into JSON, and streams lines to the server via `netcat`. No extra packages beyond what OpenWrt provides by default.

What It Does

- Listens on `br-lan` for UDP port 53
- Parses queries and answers
- Outputs newline-delimited JSON
- Streams via `nc` to `SERVER_IP:9999`

```
#!/bin/sh
SERVER_IP="YOUR_SERVER_IP"
PORT="9999"

tcpdump -i br-lan -nn -l ip and udp port 53 |
awk -v ip="$SERVER_IP" -v port="$PORT" '
/A\?/ {
 dom = $8; sub(/\.$/, "", dom)
 printf "{\"type\":\"QUERY\",\"src\":\"%s\",\"domain\":\"%s\"}\n",
$3, dom
}
/A / {
 match($0, /A .*/)
 ans = substr($0, RSTART+2, RLENGTH-2)
 printf "{\"type\":\"ANSWER\",\"src\":\"%s\",\"data\":\"%s\"}\n",
$3, ans
}' | nc $SERVER_IP $PORT
```

Server Listener & Database

SERVER/MAIN.PY | SERVER/DATABASE.PY

The server accepts raw TCP connections, accumulates bytes, splits on newline, and safely parses each JSON line. Valid events are inserted into SQLite and — for queries — enqueued for the live dashboard. A separate thread starts the Flask UI on port 5000.

Listener Loop

```
def handle_client(sock, addr):
    buffer = ""
    while True:
        chunk = sock.recv(4096).decode()
        if not chunk: break
        buffer += chunk
        while "\n" in buffer:
            line, buffer = buffer.split("\n", 1)
            data = json.loads(line)
            database.insert_data(data)
            bridge.live_traffic_queue.put({...})
```

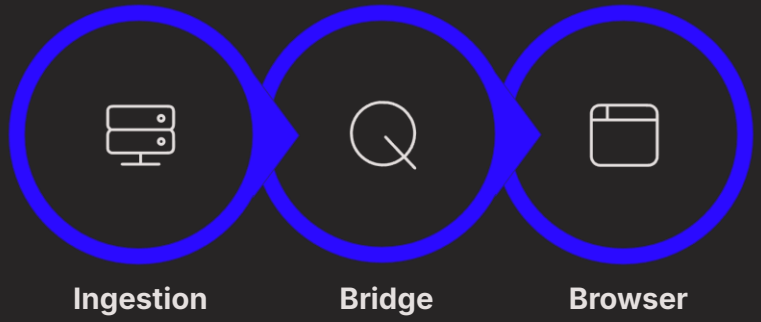
SQLite Schema

```
CREATE TABLE dns_traffic (
 id INTEGER PRIMARY KEY,
 timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
 log_type TEXT,
 src_ip TEXT,
 dst_ip TEXT,
 detail TEXT
);
CREATE TABLE blocked_domains (
 id INTEGER PRIMARY KEY,
 domain TEXT UNIQUE
);
CREATE INDEX idx_detail_src
 ON dns_traffic(detail, src_ip);
```

Live Stream via Server-Sent Events

SERVER/BRIDGE.PY SERVER/WEB_UI.PY

The `Queue` in `bridge.py` decouples ingestion from the UI thread — zero CPU polling, near-zero latency. Flask exposes `/stream` as an SSE endpoint; the browser reconnects automatically. A keepalive ping every 15 seconds prevents the browser from dropping the connection during quiet periods.



bridge.py — Queue

```
from queue import Queue
live_traffic_queue = Queue()
```

web_ui.py — SSE Generator

```
def _event_stream():
 while True:
 try:
 item = bridge.live_traffic_queue.get(
 timeout=15)
 yield f"data: {json.dumps(item)}\n\n"
 except Exception:
 yield ": ping\n\n" # keepalive
```

Web API & Blocklist Endpoints

SERVER/WEB_UI.PY

The Flask app serves the dashboard UI and a small REST API for blocklist management. The UI calls these endpoints to add or remove domains; responses indicate success, duplicates (`409`), or not-found (`404`).

GET /api/blocklist

Returns all currently blocked domains as a JSON array. Used by the dashboard to populate the blocklist table on load.

POST /api/blocklist

Accepts `{"domain": "example.com"}`. Validates, normalizes to lowercase, checks for duplicates, and inserts via `database.add_to_blocklist()`. Returns `409` if already blocked.

DELETE /api/blocklist

Removes a domain from the blocklist. Returns `404` if the domain was not found. The blocker will sync the updated list to the router on the next call.

Pushing Blocklist to the Router

SERVER/BLOCKER.PY

When the blocklist changes, `blocker.py` composes `dnsmasq` address rules and writes them to the router via SSH. A reload applies changes without dropping existing connections. The private key must be configured for passwordless SSH access.

dnsmasq Rule Format

Each blocked domain becomes a single line:

```
address=/ads.doubleclick.net/
address=/tracker.example.com/
```

dnsmasq returns `0.0.0.0` for any matching query — effectively a sinkhole.

SSH Sync Logic

```
def sync_blocklist_to_router(force=False):
 domains = database.get_all_blocked_domains()
 payload = "".join(
 f"address=/{d}/\n" for d in domains)

 # Write config file
 subprocess.run(
 base_ssh + [f"cat > {BLOCKLIST_FILE}"],
 input=payload, text=True)

 # Reload dnsmasq
 if force:
 subprocess.run(
 base_ssh + ["/etc/init.d/dnsmasq restart"])
 else:
 subprocess.run(
 base_ssh + ["/etc/init.d/dnsmasq", "reload"])
```

Running the Demo

Get the system up in minutes. If you don't have an OpenWrt router connected, use the demo populate endpoint to seed sample data and explore the dashboard immediately.

01

Start the server

```
python3 server/main.py
```

Listener binds on `0.0.0.0:9999`; Flask UI starts on `:5000`.

02

Open the dashboard

Navigate to `http://localhost:5000/` in your browser. The SSE stream connects automatically.

03

Populate demo data

```
curl -X POST
http://localhost:5000/api/demo/populate
```

Seeds sample queries and answers so you can see the UI populated without a router.

04

Block a domain

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"domain":"ads.doubleclick.net"}' \
  http://localhost:5000/api/blocklist
```

Triggers an SSH sync to the router — dnsmasq reloads and the domain is sinkholed.

05

Run the router script

```
SERVER_IP=<your_server_ip> PORT=9999 \
  sh router-script.sh
```

Run on your OpenWrt device. DNS events will appear live in the dashboard within seconds.